

CSE220 - LTI Systems & Convolution: Exam Practice Guide

0. General Process for Every Coding Question

Follow this checklist on every question, in order:

1. **Read the input format spec first.** Note exact line order (e.g. start end then values on the next line). Write a small `read_nonempty_lines + parse_signal` pair if the exam gives an input file, exactly like `main.py`'s pattern.
2. **Identify which objects you need:** is it just `DiscreteSignal` + `LTISystem`, or does it need a *new* method on `LTISystem` (deconvolution), a *new* small class (`Supersignal`), or a *plain function* that is not itself LTI (`system_b`)?
3. **Never hand-derive convolution sums.** Always express the answer using `DiscreteSignal.add`, `DiscreteSignal.multiply`, `DiscreteSignal.shift`, and `LTISystem.output / output_by_superposition`. If a formula in the problem looks like $a*T\{x_1\} + b*T\{x_2\}$, that is literally `.multiply(a).add(...)`.
4. **Compute output ranges automatically.** Never hardcode a length; use `signal.start_time + h.start_time / end_time + h.end_time` the same way `LTISystem.output_range` already does.
5. **Verify with a second method whenever the problem allows it** (two convolution routes, cumulative sum vs. convolution, network vs. `h_combined`). Report `max_absolute_difference`, which should be ~ 0 .
6. **Plot last.** Get the numeric answer correct and printed before touching `matplotlib`. Reuse `DiscreteSignal.plot` (stem plot) and the grayscale color-block pattern from Offline 1 for every visualization requirement.
7. **Print a short verification/report block** stating input range, output range, and max-diff - this is worth marks on its own in every problem.
8. **Sanity-check with the Sample I/O by hand** for at least one index before trusting the full run (e.g. compute `y[3]` yourself and compare).

Common formulas you must recognize instantly

Idea	Formula	Where it appears
Convolution sum	$y[n] = \sum_k x[k] * h[n-k]$	everywhere
Linearity test	$T\{a*x_1 + b*x_2\} = ?$ $a*T\{x_1\} + b*T\{x_2\}$	A1/A2, Problem D
Time-invariance test	$T\{x(n-k)\} = ?$ $y(n-k)$	A1/A2
Cascade	$h_{\text{cascade}} = h_1 * h_2$ (convolve)	Problem B
Parallel	$h_{\text{parallel}} = h_1 + h_2$ (add)	Problem B
Deconvolution	$h[n] = (y[n] - \sum_{k=0}^{n-1} x[k]h[n-k]) / x[0]$	Problem A, E

Binomial smoothing weight	$h[k] = C(n-1, k) / 2^{(n-1)}$	Problem C
Sparse echo IR	$h[n] = d[n] + a*d[n-D] + a^2*d[n-2D]$	Problem F
First difference / edge filter	$h[0]=1, h[1]=-1$	Problem G
Step response	$s[n] = \sum_{k \leq n} h[k]$ (cumulative sum) OR $h * u[n]$	Problem H

1. Core Helpers (reused everywhere)

```
def make_signal(start_time, end_time, values):
    """Build a DiscreteSignal from a list of values placed at
    start..end."""
    # TODO: create DiscreteSignal(start time, end time), fill with
    #       set_value_at_time; raise ValueError if len(values)
    mismatches
    pass

def signal_from_samples(start_time, end_time, samples):
    """samples: dict {time_index: value}; unspecified indices stay
    0."""
    # TODO
    pass

def max_absolute_difference(first_signal, second_signal):
    """Largest |first[n]-second[n]| over the combined time range of
    both."""
    # TODO: loop over range(min(starts), max(ends)+1)
    pass
```

2. Linearity / Time-Invariance Testers (A1/A2 style)

These MUST accept any callable apply_system (not just LTISystem.output) - do not assume .h exists on it.

```
def test_linearity(apply_system, x1, x2, a, b):
    """max| apply_system(a*x1+b*x2) -
    (a*apply_system(x1)+b*apply_system(x2)) |"""
    # TODO
    pass

def test_time_invariance(apply_system, x, k):
    """max| apply_system(x shifted by k) - (apply_system(x) shifted
    by k) |"""
    # TODO
    pass

def system_time_varying_amplifier(input_signal):
    """Non-LTI example:  $y[n] = n * x[n]$ . Exam may swap in  $x[n]^2$ ,
     $x[2n]$ , etc."""
    # TODO: build result DiscreteSignal, loop result.times(), set
    n*value
```

pass

Watch for variants: $y[n] = x[n]^2$ (fails linearity, passes TI), $y[n] = x[n-1] + 1$ (fails linearity due to the +1, passes TI), $y[n] = x[2n]$ (fails TI, an example of a *linear but time-varying* system).

3. Deconvolution / System Identification (Problem A, E)

```
def deconvolve(self, output_signal):
    """self.h is the KNOWN input x[n]; output_signal is y[n]=(x*h)
    [n].

    Recover unknown h[n], causal, length = len(y)-len(x)+1.
    h[n] = ( y[n] - sum_{k=0}^{n-1} x[k]*h[n-k] ) / x[0]
    """
    # TODO: implement iteratively for n = 0, 1, 2, ...
    pass

LTISystem.deconvolve = deconvolve # attach as a method

def polynomial_to_signal(coeffs_highest_first):
    """Highest-degree-first coefficients -> 0-indexed causal
    DiscreteSignal."""
    # TODO: reverse list, build DiscreteSignal from n=0
    pass

def signal_to_polynomial(signal):
    """Inverse of polynomial_to_signal."""
    # TODO
    pass
```

Assumption to state in your report: $x[0]$ must be nonzero (division), and x must be causal (starts at $n = 0$) or you must shift it to be so first.

4. Cascade + Parallel Networks (Problem B)

```
x -> [h1] --\
           +--> (add) --> [h3] --> y
x -> [h2] --/
```

```
def combined_impulse_response(h1, h2, h3):
    """h_combined[n] = (h1[n] + h2[n]) * h3[n]"""
    # TODO: DiscreteSignal.add then LTISystem(sum).output(h3) --
    careful,
    #          convolving two IMPULSE RESPONSES uses the same
    LTISystem.output
    #          machinery: treat one as the "input" to a system built
    from the other
    pass

def run_network_the_long_way(x, h1, h2, h3):
    """Simulate each stage: parallel h1,h2 on x, add, cascade
    through h3."""
    # TODO
    pass
```

Common mistake: convolving $h_1 * h_2 * h_3$ in a straight line describes a pure cascade network, NOT this parallel-then-cascade network. Always re-derive the block diagram algebraically before coding.

5. Binomial-Weighted Smoothing (Problem C)

```
import math

def binomial_impulse_response(window_size):
    """h[k] = C(window_size-1, k) / 2**(window_size-1),
    k=0..window_size-1.
    k=0 is the NEWEST sample's weight."""
    # TODO: use math.comb; assert sum(h.values) is close to 1.0
    pass

def trim_partial_window_edges(signal, window_size):
    """Drop the (window_size-1) partially-overlapped samples at each
    end,
    same as a plain moving average would."""
    # TODO
    pass
```

6. Multi-Echo Superposition (Problem D)

```
class Supersignal:
    """Several DiscreteSignal components at their own time
    ranges."""

    def __init__(self, components):
        # TODO: store list
        pass

    def combined_input(self):
        """Fold all components together with .add()."""
        # TODO
        pass

    def output_super_via_combined_input(system, supersignal):
        """Sum components first, convolve once."""
        # TODO
        pass

    def output_super_via_per_component_sum(system, supersignal):
        """Convolve each component separately, then add the outputs."""
        # TODO: outputs will have different ranges -- DiscreteSignal.add
        handles that
        pass
```

This is the superposition principle demonstrated **on the input side** - state that explicitly in your report.

7. Digital Echo / Reverb (Problem

F)

```
def echo_impulse_response(delay_D, alpha):  
    """h[n] = delta[n] + alpha*delta[n-D] + alpha^2*delta[n-2D],  
    length 2*D+1, mostly zeros."""  
    # TODO  
    pass
```

Do not special-case the sparsity anywhere else - LTISystem.output should convolve it exactly like any dense impulse response.

8. 1-D Edge Detector (Problem G)

```
def first_difference_impulse():  
    """h[0]=1, h[1]=-1 (reuse Offline-1 built-in filter #6)."""  
    # TODO  
    pass  
  
def detect_edges(output_signal, threshold):  
    """Indices n where |y[n]| > threshold."""  
    # TODO  
    pass
```

Decide (and justify in your report) whether the boundary “edge” at $n=0$ (the implicit-zero-before-the-signal transition) should be reported.

9. Step Response Recovery (Problem H)

```
def step_response_cumulative_sum(h):  
    """Method 1: s[n] = s[n-1] + h[n], no convolution."""  
    # TODO  
    pass  
  
def step_response_via_truncated_convolution(h, U):  
    """Method 2: u[n]=1 for n in [0,U] (finite truncation), s2 =  
    h*u."""  
    # TODO: build u as DiscreteSignal(0, U) all ones, then  
    LTISystem(h).output(u)  
    pass
```

Explain why $s2[n]$ diverges from the true step response for n near U : the true $u[n]$ is infinite to the right; any `DiscreteSignal` you build is necessarily finite, so once the window of h slides past U the convolution sum silently loses terms.

10. Visualization Toolkit (matplotlib + numpy)

```
import numpy as np  
import matplotlib.pyplot as plt  
  
def plot_three_signals_as_stems(input_signal, impulse_response,  
                                output_signal,  
                                save_path=None):  
    """3 stacked stem subplots: x[n], h[n], y[n]."""
```

```

# TODO:
# fig, axes = plt.subplots(3, 1, figsize=(10, 8),
constrained_layout=True)
# input_signal.plot("Input x[n]", ax=axes[0])
# impulse_response.plot("Impulse response h[n]", ax=axes[1])
# output_signal.plot("Output y[n]", ax=axes[2])
# if save_path: fig.savefig(save_path, dpi=150)
pass

def normalized_grayscale_rgb(signal):
    """Map signal values to 0..255 grayscale, mid-gray if signal is
    constant."""
    # TODO:
    # values = np.array(signal.values, dtype=float)
    # lo, hi = float(np.min(values)), float(np.max(values))
    # scale to 0..255 with np.round(...).astype(np.uint8)
    # np.stack([gray, gray, gray], axis=-1)
    pass

def plot_signal_as_color_blocks(signal, title, ax):
    """Single-row imshow, white cell boundaries, n tick labels."""
    # TODO:
    # rgb = normalized_grayscale_rgb(signal); image = rgb.reshape(1,
len(rgb), 3)
    # ax.imshow(image, aspect="auto", interpolation="nearest")
    # ax.set_xticks(range(len(signal)));
ax.set_xticklabels(list(signal.times()))
    # for b in range(len(signal.values)+1): ax.axvline(b-0.5,
color="white", lw=0.8)
    pass

def plot_before_after_color_blocks(before_signal, after_signal,
save_path=None):
    """2 stacked subplots (used for edge-detector before/after)."""
    # TODO:
    # fig, axes = plt.subplots(2, 1, figsize=(10, 3.6),
constrained_layout=True)
    # plot_signal_as_color_blocks(before_signal, "Before", axes[0])
    # plot_signal_as_color_blocks(after_signal, "After", axes[1])
    # if save_path: fig.savefig(save_path, dpi=150)
    pass

```

Matplotlib cheat-sheet for this exam:

- `fig, axes = plt.subplots(n_rows, 1, figsize=(w, h), constrained_layout=True)`
◦ always use this for “N separate stem plots” requirements.
- `ax.stem(time_values, values)` - manual stem plot if not using `.plot()`.
- `ax.axhline(0, color="black")` - zero reference line.
- `ax.set_xticks(...)`, `ax.set_xticklabels(...)` - needed when the color-block x-axis must show actual n values, not array indices.
- `ax.imshow(image, aspect="auto", interpolation="nearest")` - for the grayscale color-block visualization; image must be shape (1, L, 3).
- `fig.savefig(path, dpi=150)` then `plt.close(fig)` to avoid figure buildup across multiple problems in one script.

Numpy cheat-sheet:

- `np.array(list, dtype=float)`, `np.min`, `np.max`, `np.isclose`
 - `np.round(...).astype(np.uint8)` for grayscale conversion
 - `np.stack([a, a, a], axis=-1)` to make a 3-channel grayscale image
 - `np.ceil`, slicing `arr[::step]` - used in `readable_time_ticks` style helpers when a signal is too long to label every sample
-

11. Full Skeleton File (same as the exam template - do not skip re-typing)

signatures, they are worth marks even before logic)

```
"""
CSE220 - LTI Systems & Convolution - MASTER PRACTICE TEMPLATE.
Rules: only DiscreteSignal / LTISystem; no numpy.convolve /
scipy.signal;
numpy/matplotlib allowed for arrays and plots only.
"""

import math
import numpy as np
import matplotlib.pyplot as plt
from signal_lti import DiscreteSignal, LTISystem


def make_signal(start_time, end_time, values):
    # TODO
    pass


def signal_from_samples(start_time, end_time, samples):
    # TODO
    pass


def max_absolute_difference(first_signal, second_signal):
    # TODO
    pass


def print_signal(signal, name):
    # TODO
    pass


def test_linearity(apply_system, x1, x2, a, b):
    # TODO
    pass


def test_time_invariance(apply_system, x, k):
    # TODO
    pass


def system_time_varying_amplifier(input_signal):
    # TODO
    pass


def deconvolve(self, output_signal):
    # TODO
    pass


LTISystem.deconvolve = deconvolve


def polynomial_to_signal(coeffs_highest_first):
    # TODO
    pass
```

```
def signal_to_polynomial(signal):
    # TODO
    pass

def combined_impulse_response(h1, h2, h3):
    # TODO
    pass

def run_network_the_long_way(x, h1, h2, h3):
    # TODO
    pass

def binomial_impulse_response(window_size):
    # TODO
    pass

def trim_partial_window_edges(signal, window_size):
    # TODO
    pass

class Supersignal:
    def __init__(self, components):
        # TODO
        pass

    def combined_input(self):
        # TODO
        pass

def output_super_via_combined_input(system, supersignal):
    # TODO
    pass

def output_super_via_per_component_sum(system, supersignal):
    # TODO
    pass

def echo_impulse_response(delay_D, alpha):
    # TODO
    pass

def first_difference_impulse():
    # TODO
    pass

def detect_edges(output_signal, threshold):
    # TODO
    pass

def step_response_cumulative_sum(h):
    # TODO
    pass

def step_response_via_truncated_convolution(h, U):
    # TODO
    pass
```

```

def plot_three_signals_as_stems(input_signal, impulse_response,
                                output_signal,
                                save_path=None):

    # TODO
    pass

def normalized_grayscale_rgb(signal):

    # TODO
    pass

def plot_signal_as_color_blocks(signal, title, ax):

    # TODO
    pass

def plot_before_after_color_blocks(before_signal, after_signal,
    save_path=None):

    # TODO
    pass

def main():
    # TODO: wire up whichever sample problem you are practicing
    today,
    #      using the Sample I/O tables from the practice-problem
    PDF.

    pass

if __name__ == "__main__":
    main()

```

12. Sample I/O Quick Reference (for self-checking, values only)

Problem	Key inputs	Expected key output
A: Deconvolution	x=[1,2,-1] n=0..2; y=[2,3,-3.5,5,5.5,-3] n=0..5	h = 2, -1, 0.5, 3
B: Cascade+Parallel	x n=-1..4; h1=[1,1,1]; h2=[1,0,-1]; h3=[0.5,0.5]	h_combined = 1.0,1.5,0.5,0.0
C: Binomial smoothing	prices=[4,8,6,10,14,12,16,20], n=4	weights .125,.375,.375,.125; first out 7.00
D: Multi-echo	h=[1,0.5,0.25]; 3 components	y[-1..5] = 3, -1.5, 5.25, 4.25, 3.5, 1.0, 0.25
E: Poly division	dividend [2,3,1,6]; divisor [1,2]	quotient [2,-1,3]
F: Echo/reverb	x n=0..9; D=3; alpha=0.5	y[3] = 1.0
G: Edge detector	row of 14 pixels; threshold=30	edges at n=0,3,7,11,14
H: Step response	h n=-1..2 = 3,-1,2,0.5; U=20	s[-1..8] = 3,2,4,4.5,4.5,...

Use these to sanity-check your own implementation before trusting a full run.